AMD Instinct™ MI300 GPU Memory Bandwidth

XIAOMIN LU, Advanced Micro Devices, Inc., USA

This white paper discusses the memory bandwidth attainment on AMD Instinct™ Graphics Processing Units(GPUs) with primary focus on MI300 family SoCs. It aims to illustrate the GPU performance model related to memory accesses, the inter-play between software and hardware, and the impact of hardware architecture and micro-architecture on the achievable memory bandwidth. The performance analysis is based on the GFX9 architecture and the methodology herein is equally applicable to MI200, MI300, MI308, and MI350 silicons with minor silicon specific parameter adjustment.

Additional Key Words and Phrases: Graphics Processing Unit (GPU), Silicon-on-Chip (SoC), MI300A, MI300X, MI308, MI350, High-bandwidth Memory(HBM), Performance Model, Performance Analysis Methodology

ACM Reference Format:
Xiaomin Lu. 2024. AMD Instinct™ MI300 GPU Memory Bandwidth. 1, 1 (June 2024), 30 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

AMD Machine Instinct GPUs[8] have enjoyed wild success in high performance computing (HPC), artificial intelligence and machine learning (AIML), and gaming applications. The Frontier supercomputer, powered by over thirty-seven thousand MI250 GPUs, continues to dominate the TOP500 as the first supercomputer reaching the Exascale computing realm[5][9]. The next-generation MI300 GPU will drive the El Capitan supercomputer to reach two exaflops of double precision computing power[6]. In addition to the massive compute power, the offered HBM bandwidth also leaps for each silicon generation. For example, MI200 provides 3.2TB/s bandwidth, MI300 and MI308 make available 5TB/s via 8 HBM stacks, while MI350 offers a sheer 8TB/s raw bandwidth with the same number of stacks, a 60% jump from the previous generation silicon. Apart from the aggregated achievable HBM bandwidth, the latest large language model computing increasingly demands strong scaling in HBM bandwidth attainment, i.e., the achievable memory bandwidth on per Compute Unit(CU) basis. Combined with dynamic power managemen(DPM), which often results in lower graphics compute frequencies for bandwidth limited workloads, fully utilizing the massively available HBM bandwidth and enabling high performance compute for both HPC and ML workloads is becoming a truly challenging task. Development of high performing workloads achieving the best HBM bandwidth requires not only deep understanding of GPU software architecture and the intricacy of matrix multiplication implementation, but also the firm grasp of the GPU hardware architecture and micro-architecture details. The interplay between software and hardware imposes important, sometimes mysterious, impact on the eventually measured memory bandwidth.

Fundamentally, the HBM bandwidth achieved by a workload, whether at per-CU level or system level, is limited by the underlying hardware architecture (roofline), and the software capability of issuing enough memory requests to hide the memory latency. The memory bandwidth-latency dynamics also plugs into the feedback loop and affects the system performance in an important way. In addition, the achievable bandwidth is unavoidably impacted by the inherent hardware

Author's address: Xiaomin Lu, xiaomin.lu@amd.com, Advanced Micro Devices, Inc., 7171 Southwest Parkway, Austin, Texas, USA, 78735.

2024. This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in , https://doi.org/10.1145/nnnnnnn.nnnnnnn.

Table 1. MI GPUs Compute vs. HBM Bandwidth

SoC	Compute Units (CUs)	HBM Bandwidth (TB/s)
MI250X	220	3.2
MI300X	304	5.1
MI308	80	5.1
MI350	256	8.1

micro-architecture implementation. The solid understanding of the details is critical for productive workload performance analysis and optimizations.

This white paper aims to shed some light on the MI300 performance model and actionable analysis methodology to help ease the challenge. Unless otherwise noted, the term MI300 refers to all silicons in the same family, including MI300A, MI300X, MI308, and MI350.

The rest of the paper is organized as follows. Section 2 provides an overview of the MI300 SoC system architecture and the theoretical roofline bandwidths at each memory hierarchy. Section 3 covers the software attainable memory bandwidths. It details the software visible performance model for each major component, including frontend wave dispatch, compute unit, Vector Level-1 Data Cache (vL1D) cache, and Level-2 Cache (L2) cache. Section 4 describes the actionable performance analysis methodology. It is built upon the high level GPU performance models and the performance events available from ROCProfiler[4]. Some practical analysis recipes are also provided to allow systematic workload characterization and bottlenecks identification by using key performance events and metrics accessible to most developers.

2 MI300 THEORETICAL BANDWIDTH

2.1 Performance Architecture

The high-level MI300 performance architecture is shown in Figure 1. MI300 is built off multi-die chiplet SoC architecture [1]. An SoC is configured with multiple Active IO Dies (AIDs), each hosting 1-2 Accelerator Complex Dies (XCDs) and system infrastructure blocks. All dies are interconnected with AMD Infinity Fabric™ and connect to 8 stacks of High Bandwidth Memory (HBM). In addition, AMD Infinity Cache™ is also enabled between the data fabric and HBM, acting as the non-coherent Last Level Cache (LLC).

Each XCD contains a cluster of 4 Shader Engines (SEs) sharing a 16-channel L2 cache with hardware enforced coherency. The SE provides the main compute horsepower via SoC specific, configurable number of Compute Units (CUs), each having a dedicated vL1D.

The CU is a building block for 64-thread wavefront computing. Each CU is constructed with four Single Instruction Multiple Data (SIMD) units, one Scalar ALU (SALU) unit, one SeQuencer (SQ), one Local Data Share (LDS) and the associated Vector GPRs (VGPRs) and Scalar GPRs (SGPRs). The SIMD unit enables both vector and matrix operations for various integer and floating-point data formats, fully supporting Machine Learning (ML) and High Performance Computing (HPC) applications. Each SIMD unit is shared by 8 waves to allow better memory latency hiding and higher compute efficiency. The SALU unit implements scalar operations shared by all threads in the wavefront. Each CU contains 32-lane Static Random Access Memory (SRAM) based LDS to allow data sharing among waves in a workgroup. The SQ manages all wave slots and the resources (VGPR, SGPR, LDS, etc). It interfaces with Shader Processor Input (SPI) and is responsible for wave dispatch, execution, and wave retirement. At each cycle, SQ can issue up to 5 classes of instructions with all dependencies resolved, achieving a max Instructions Per Cycle (IPC) of 5. The

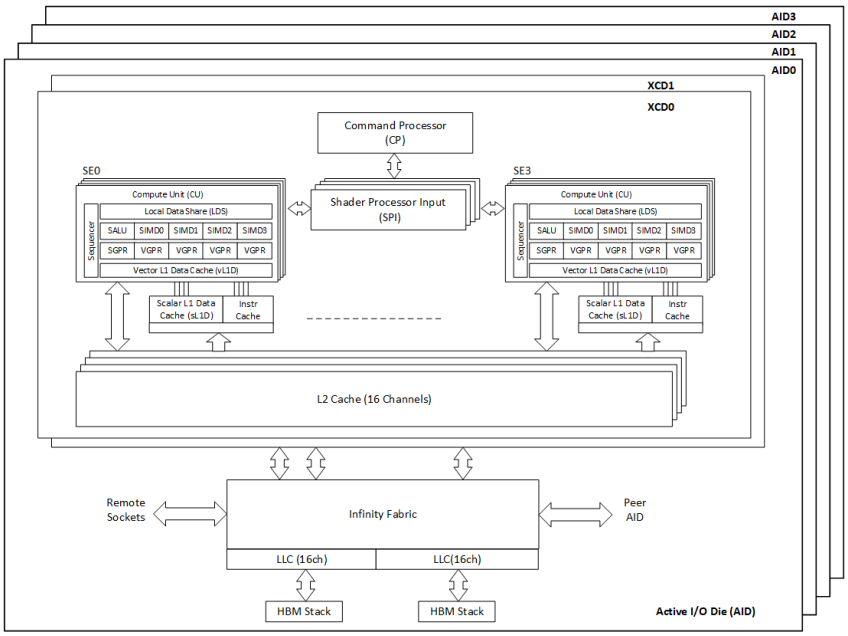


Fig. 1. MI300 Performance Architecture

5 pipelines execute Vector ALU (VALU), SALU/Scalar Memory (SMEM), LDS, branch/control, and Vector Memory (VMEM) instructions in parallel . The instruction issue rates for SALU/SMEM and branch instructions are 1 cycle/instruction, compared to 4 cycles, or Quad Cycle (QCYCLE), for VALU and VMEM instructions. The VMEM pipeline is shared by all 4 SIMDs to enable all external memory accesses, including HBM, I/O, and host memory.

On the frontend, each XCD contains a Command Processor - Compute (CPC). The CPC communicates with CPCs on other XCDs to implement coordinated workgroup dispatch at granularity of configurable chunk sized of workgroups. The kernel dispatched from the host side is broken into chunks of workgroups, which are allocated to XCD in round-robin fashion. CPC is responsible for unrolling the chunks of workgroups and dispatching them to all 4 SEs on the same XCD, also in round-robin. The workgroup dispatch rate to an SE is 4 cycles/workgroup.

Each SE contains an SPI to manages wave dispatch, wave state and resource tracking, and wave retirement. Upon receiving workgroups from CPC, it tries to unroll them into waves, considering the available resources and the incoming workgroup's resource requirements. A workgroup's execution resources include wave slot, VGPR, SGPR, LDS, barriers, etc. A workgroup is ready to be unrolled and dispatched to CU when the resource requirements for all waves in the workgroup are satisfied. The peak wave dispatch rate is 4 cycles/wave. In addition to the workgroup unrolling and wave dispatch, SPI also tracks wave states and events to ensure orderly execution.

Table 2 lists the key architectural parameters for existing MI300 family SoCs.

Table 2. MI300 GPU System Architecture and Theoretical Performance

	Architecture **GPU Chip**	**Unit**	CDNA3 MI300A	CDNA3 MI300X	CDNA3 MI308	CDNA3 MI350[1]
Clocking[2]	SCLK	MHz	1300	1300	1420	1000
	UCLK	MHz	1250	1250	1250	1000
Compute Configuration	AIDs		4	4	4	4
	XCDs		6	8	4	8
	SEs/XCD		4	4	4	4
	CUs/XCD		38	38	20	32
	Total CUs		288	304	80	256
CU Configuration	SIMDs		4	4	4	4
	LDS	KB	64	64	64	128
	Wavefront Size	Threads	64	64	64	64
	Max workgroup Size		2048	2048	2048	2048
	Max Waves per CU		32	32	32	32
	SIMD Pair-LDS Bus Width	Bytes	64	64	64	128
	CU-vL1D Bus Width	Bytes	64	64	64	64
Vector-L1 Cache Configuration	Cacheline Size	Bytes	128	128	128	128
	Capacity	KB	32	32	32	32
	Sets		4	4	4	4
	Associativity	Ways	64	64	64	64
	vL1D-L2 Read Bus Width	Bytes	128	128	128	128
	vL1D-L2 Write Bus Width	Bytes	64	64	64	64
L2 Cache Configuration	Cacheline Size	Bytes	128	128	128	128
	Capacity	MB	4	4	4	4
	Channels		16	16	16	16
	Sets/channel		128	128	128	128
	Associativity	Ways	16	16	16	16
	L2-EA Read Bus Width	Bytes	64	64	64	64
	L2-EA Write Bus Width	Bytes	64	64	64	64
LLC Configuration	Cacheline Size	Bytes	64	64	64	64
	Capacity	MB	256	256	256	256
	Channels		128	128	128	128
	Sets/channel		2048	2048	2048	2048
	Associativity	Ways	16	16	16	16
	LLC-UMC Bus Width	Bytes	64	64	64	64
HBM Configuration	Stacks		8	8	8	8
	Type		HBM3	HBM3	HBM3	HBM3
	Interface	Bits	8192	8192	8192	8192
	Capacity	GB[3]				
	Bandwidth	TB/s				

2.2 Theoretical Bandwidth

MI300 is a highly complex parallel compute system with many levels of caches matched with high bandwidth HBM. To evaluate the theoretical, as well as software attainable, memory bandwidths,

[2]Reference numbers only. The actual operating frequencies on an SoC may vary.
[3]All bandwidth units are metric, i.e., 1GB=10^9 bytes and 1TB=10^{12} bytes.

it is appropriate to check the bandwidth at each system hierarchy level to provide crystal view. These bandwidth figures are essential to check a workload's speed-of-light performance. The levels of hierarchy include:

(1) per-CU bandwidth
(2) per-LDS bandwidth
(3) per-vL1D bandwidth
(4) per L2 bandwidth
(5) per-XCD bandwidth
(6) per-HBM stack bandwidth
(7) Aggregated CU bandwitdh
(8) Aggregated vL1D bandwidth
(9) Aggregated L2 bandwidth
(10) Aggregated LLC bandwidth
(11) Aggregated HBM bandwidth

The MI300 family silicon's theoretical bandwidth is determined by its SoC architecture, bus sizing, silicon specific configurations, and clocking. At per-CU level, one VMEM pipeline interfaces with vL1D cache via a 64-byte data bus, achieving a throughput of 64 bytes/cycle. Each CU contains an on-chip LDS built as dual-port SRAM. Each port is shared by a SIMD pair with a 64-byte (MI300A/MI300X/MI308) or 128-byte bus (MI350). The aggregated LDS bandwidths for all 4 SIMDs are 128 bytes/cycle (MI300A/MI300X/MI308), and 256 bytes/cycle (MI350), respectively.

The vL1D cache is built with 4 tagrams, each is configured as physically indexed, physically tagged, 64-way set associative cache with cacheline size of 128 bytes. The total capacity of the vL1D cache is 32KB. The max tagram lookup rate is 4 lookups/cycles, one for each tagram. The vL1D interfaces with the CU via a 64-byte bus, achieving max read or write bandwidth of 64 bytes/cycle. All vL1D miss traffic are directed to L2 cache. The vL1D-L2 read and write buses are 128 bytes and 64 bytes, respectively, achieving the max read throughput of 128 bytes/cycles and max write throughput of 64 bytes/cycles.

At per-XCD level, each XCD contains an L2 cache, serving all clients', including CU's, memory requests from all SEs. The L2 cache is built with 16 independent channels, each is configured as 128-set 16-way set-associative cache with cache line size of 128 bytes. The aggregated L2 cache capacity is thus 4MB per XCD. The CU requests, as well as other clients of L2, including Sequencer Cache (SQC), Unified Translation Cache - Level 2 (UTCL2), etc, are routed to one of the L2 channel instances via a 16x16 interconnect network, each bus is 128-byte wide. Thus, the maximum L2 read bandwidth is 2048 bytes/cycle, given the 128-byte max read request size. The max write request size is 64-byte, resulting in the peak L2 write bandwidth of 1024 bytes/cycle. On the SoC interface side each L2 channel is connected to the Infinite Fabric via Efficiency Arbiter (EA) using a 64-byte bus, The aggregated XCD-Fabric bandwidth is 1024 bytes/cycle, limited only by the GL2 cache interface bus width.

On SoC side, the MI300 series SoCs are equipped with 8-stack Gen3 HBMs. Each HBM stack is controlled by 16 Unified Memory Controllers (UMCs) channels. MI300A/MI300X/MI308 UMCs operate at 1.25GHz and interfaces with HBM PHY with 32-byte bus, the theoretical HBM bandwidth becomes 5.12 TB/s[4]. MI350's UMC-PHY interface is built with 64-byte bus operating at 1.0GHz. It thus enjoys 60% HBM bandwidth improvement, achieving 8.1 TB/s.

Table 3 lists the MI300 theoretical per-CU, per-XCD, and HBM bandwidths under nominal graphics and memory operating frequencies. The SOC level theoretical bandwidth is thus

[4]8 Stacks × 16 UMCs/stack × 32bytes/UMC * 1.25GHz = 5120 GB/s

Table 3. MI300 Theoretical Bandwidths

Hierarchy Unit	Interface BW Bytes/cycle	MI300X GB/s	MI308 GB/s	MI350 GB/s
SCLK		1.3 GHz	1.42 GHz	1.0 GHz
UCLK		1.25 GHz	1.25 GHz	1.0 GHz
#CU		304	80	256
#XCD		8	4	8
per-CU	64	83.2	90.9	64
per-XCD	1,024	1,331	1,452	1,024
CU (aggr)		25,293	7,270	16,384
XCD (aggr)		10,650	5,816	8,192
HBM	32	5,120	5,120	8,192

$$\text{SoC_BW} = min\{\text{CU_BW, XCD_BW, LLC_BW, HBM_BW}\} \tag{1}$$

3 MI300 ATTAINABLE BANDWIDTH

Compared to micro-benchmark workloads, which are purposely designed to stress HBM or a cache hierarchy and are relatively easy to analyze, real-world workload performance is a complex topic requiring systematic exploration. The achieved figure of merit is the result of interplay among software design and optimization, hardware architecture configurations, micro-architecture constraints, and even dynamic power management. There are always bottlenecks for a workload under test on a platform, except that the bottlenecks move around as optimizations proceed further. The systematic coverage of performance analysis deserves a book to thoroughly expose every area, this document merely focuses on the graphics compute side topics related to the bandwidth attainment. The SoC side bandwidth-latency dynamics is treated as a "black box." In addition, the document assumes fixed frequency, leaving power management related topics to other discussions.

At high level, the GPU subsystem, including compute and memory, can be roughly viewed as an open queueing network. The system is interconnected by a network of frontends, compute units, cache instances at each hierarchy, data fabrics, memory controllers, and the memory device. A rigorous mathematical modeling and analysis of such complex network is probably beyond being feasible, except for academic importance. Nevertheless, a queueing model based performance analysis methodology provides insight into the impact of various IP components on the system performance. It provides guidance to fine tune software and hardware parameters, e.g., FIFO sizing, workgroup sizing, etc. The queueing model approach is also the basis to systematically define hardware perfmon events.

The key to achieving high memory bandwidth is memory latency hiding. This demands the workload to issue enough memory request to fill the deep "memory pipe." At the highest level, the intriguing bandwidth optimization can be explained by Little's Law in queueing theory, namely, $N = \lambda T$, where N is the system occupancy, λ is the throughput, and T is the workload dwelling time. In order to achieve higher throughput, one should drive up the system occupancy, or lower the wavefront life. For GPU compute, the queueing occupancy is not just the wave occupancy, but the aggregated number of memory requests dwelling in the system.

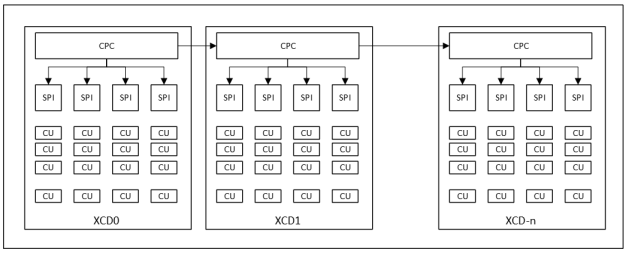


Fig. 2. MI300 multi-die compute architecture

Both system occupancy and workload dwelling time are heavily affected by both software and hardware implementation. The optimized workload performance is the result of inter-play among the following factors:

- Software design and optimization,
- Frontend workgroupd and wavefront dipatch,
- Compute Unit instruction scheduling and issueing,
- Vector L1 cache,
- Vector L2 cache,
- SoC bandwidth-latency dynamics

The optimization for both queueing occupancy and workload dwelling time are detailed in following subsections.

3.1 Frontend Workgroup and Wave Dispatch

3.1.1 Workgroup Dispatch. The MI300 SoCs are built off multi-die architecture, as shown in Figure 2. Each XCD contains a CPC. It coordinates with CPCs on other XCDs to implement cooperative workgroup dispatch and to enforce ordered workgroup execution.

Kernels dispatched by the host are chopped into chunks of workgroups, each chunk consisting of C workgroups. Each CPC "walks" through the stream of chunks and picks the chunks in round-robin. For example, CPC#0 takes chunk index $8k, k = 0, 1, 2, ...$, in an 8-XCD MI300 SoC. The CPC walk rate is 1 workgroup/cycle. The period to select the candidate chunk, C_p, is

$$C_p = \#\text{XCD} * C \text{ (cycles)}$$

The CPC also dispatches the assigned workgroups to the 4 SPIs in round-robin. From SPI perspective, the amount of work it receives from a chunk is $C/\#SPI$. Assuming the workgroup size is WG waves, it takes a total of C_s cycles to finish the wave dispatch, where

$$C_s = \frac{C}{\#SPI} * WG * 4,$$

since the SPI's wave dispatch rate is 4 cycles/wave. The multi-die system will result in workgroup dispatch bound when $C_s < C_p$, i.e., a CPC has idle time until the next chunk is available. Therefore, dispatch bottleneck condition is

$$\frac{C}{\#SPI} * WG * 4 < \#\text{XCD} * C,$$

or

$$WG < \#\text{XCD}.$$

Table 4. MI300 SPI Resource Allocation and Granulatiry

Resource	Scope	Capacity	Granulatiry
VGPR	SIMD	512	16
SGPR	SIMD	112	8
Wave Slot	SIMD	8	1
Bulky Workgroup	CU	1	1
LDS	CU	64KB	512
Barrier	CU	32	1
Named Barrier	CU	64	4
Scratch Ring	SE	(config)	1

Essentially, an MI300 system will incur CPC dispatch limit if the workgroup size is less than 8 waves.

3.1.2 Wave Dispatch. The unroll of workgroups into waves and wave dispatches to CU is done by SPI. Figure 3 depicts the high-level SPI performance model. SPI manages all CU resources and monitors wave states and events. All required resources by the workgroup have to be satisfied before any of the waves can be dispatched. Table 4 lists the SPI managed resources and the allocation granularity. The CU wave occupancy is the maximum number of waves a CU can accommodate, subject to all the resource constraints. For example, for a kernel configured with 4 waves/workgroup requiring 32KB LDS memory, the max wave occupancy per CU is 8. Only two workgroups can run on a CU without violating the LDS resource constraint.

Similar to the workgroup dispatch bound, wave dispatch bound could occur if a wave's execution time, T_s, is so short that it finishes before SPI can dispatch the next wave to fill its spot. From a wave slot perspective, the dispatch period is a result of round-robin work distribution on CPC, SPI, and CU. Assuming the workload occupancy is W_o, satisfying the static resource constraints, the wave dispatch period, T_p, is

$$T_p = C * \#\text{XCD} * \#\text{SE/XCD} * \#\text{CU/SE} * \frac{W_o}{WG * C} \quad (2)$$

$$= \frac{\#\text{TotalCUs} * W_o}{WG}, \quad (3)$$

where #TotalCUs is the total amounts of CUs enabled by an SOC. A workload becomes wave dispatch bound when $T_s < T_p$. For example, if a kernel running on MI300X is configured with 4 waves/group and max static wave occupancy of 32, it becomes wave dispatch bound when $T_s < 304 * 32/4 = 2432$ cycles.

3.2 Compute Unit

The high-level CU performance model is shown in Figure 4. A single VMEM pipeline services memory requests from active waves on all four SIMD units. The VMEM pipeline interfaces with Vector L1 cache via three separate buses: command, address, and data, respectively. The command and address FIFO each has a depth 48, while data FIFO is 64-deep to accommodate the write data to memory. MI300 employs a 4-phase design, which serves 16 threads per phase. As such, the VMEM pipeline can pass the whole instruction information to the Vector L1 cache in 4 cycles, assuming no back pressure from the downstream cache. In the case of backpressure, the VMEM instruction issue will be spaced out, further increasing wave life and resulting in lower per-CU bandwidth.

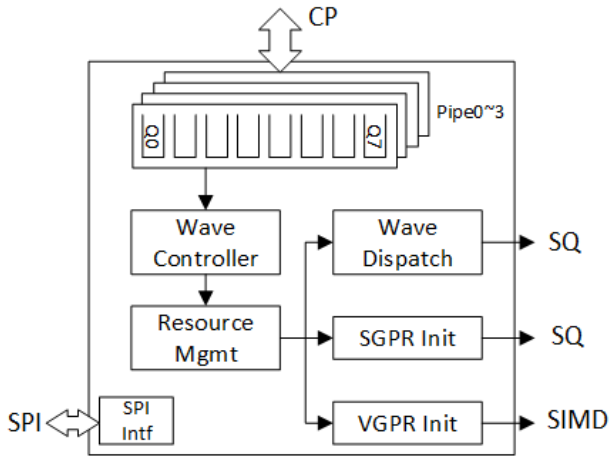


Fig. 3. MI300 SPI Performance Model

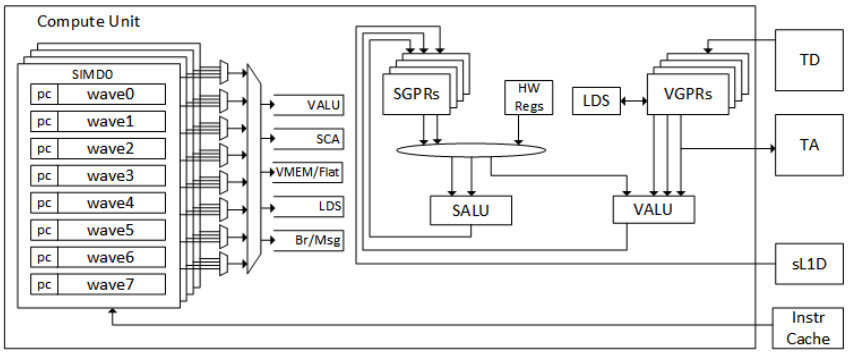


Fig. 4. MI300 CU Performance Model

At wavefront level, one needs to increase the wavefront occupancy in order to drive up the aggregated memory request rate at per-CU basis. On the other hand, per-wavefront thread shall boost the memory request rate via issuing more memory accesses during the wave execution. Given that compute and memory accesses are handled by separate hardware pipelines, higher efficiency can also be achieved via overlapping more compute with memory accesses. The L1/L2 cache subsystems and SoC fabric and memory subsystem also play a vital role in affecting the system performance. Lower memory latency results in the lower wavefront life and higher throughput.

The theoretical per-CU memory Read/Write bandwidth is limited by the CU-GL1 data bus, which is 64-byte wide, achieving a maximum of 64 bytes/cycle. The actual achieved workload bandwidth is determined by the dynamics of the wavefront dispatch, execution, and retirement, which can be modeled as point random process.

The wave dispatch interval, defined as the duration between the end of the previous wave and the start of the next wave, denoted as T_d. Each wave has a lifespan of T_s, defined as the duration between the cycle the wave is instantiated and the cycle the wave is retired. The wave life is highly affected by memory access latency, barrier synchronization, and other factors. This paper

primarily considers the memory latency. Each wave's memory access characteristics is defined by the workload and the compilers. It can be described by the number of memory instructions per wave, n, and the request size of each instruction, R, in unit of bytes. The wave occupancy, W_o, is average number of active waves in a compute unit in equilibrium. It is the result of interaction between frontend wave dispatch, wave computing, and memory accesses.

The steady-state per-CU bandwidth, in unit of bytes/cycle, can be calculated by

$$BW_{cu} = \frac{W_o * n * R}{T_d + T_s}, \tag{4}$$

where $T_d + T_s$ is the overall wave life considering both frontend dispatch and active wave execution. The instruction request size, R, is determined by per-thread request size. For example, the buffer_load_dwordx2 instruction has request size $R = 512$ bytes. Considering that the per-CU bandwidth is ultimately limited by the return data bus width of 64-byte from the vector L1 cache, the effective per-CU bandwidth can be described as

$$BW_{cu} = min\{\frac{W_o * n * R}{T_d + T_s}, 64\} \tag{5}$$

Equation (5) clearly shows the impact of hardware architecture and software design on the achievable per-CU bandwidth. The maximum per-CU bandwidth is limited to 64 bytes/cycle. To increase the per-CU bandwidth, one should increase the wave occupancy, the number of memory requests per wave, or the request size. The wave occupancy is determined the per-workgroup and per-wave resource usage determined by software design, and the available per-CU hardware resources. The affecting resources include VGPR, LDS, barrier, wave slots, etc. The per-CU bandwidth can also improve if the overall wave latency is decreased. This can be achieved via reducing the wave dispatch latency T_d, or the active wave life T_s. T_d is heavily affected by the frontend workgroup and wave dispatch efficiency. T_s is dictated by the wave occupancy and the memory bandwidth-latency dynamics perceived by the compute unit. Longer wave life could result from the higher wave occupancy, due to the VMEM access sharing between all waves on a compute unit.

3.3 Texture Addressing Unit (TA)

The MI300 Vector-L1 cache subsystem consists of three main components: TA, Texture Data Unit (TD), and the cache controller(Texture Cache per Pipe, also known as vector L1 Cache (TCP)), as shown in Figure 5. TA is an interface block between CU and TCP. It accepts memory instructions at a rate of 4 cycles/instruction from CU, and stores them in the internal buffer. TA converts the received instructions into a format compatible with TCP and passes down the request, at a rate of 4 unique addresses per cycle, for cache lookup. In the case of internal buffer build-up, TA backpressures CU to slow down further memory accesses. On the cache lookup return path, TD re-assembles cache line lookup data from TCP and returns them to CU.

One important processing TA performs is called address coalescing. TA is capable of passing 4 unique thread addresses/cycle to TCP. For non-coalesced thread addresses, a total of 16 cycles is needed to pass all 64 thread addresses for each wavefront. For coalescible instructions, TA tries to identify coalescible wavefront addresses, where a sequence of up to 4 contiguous accesses can be encoded using a single address. In the best case scenario, each address passed to TCP represents 4 threads' worth information. Given that TA carries 4 addresses per cycle to TCP, only 4 cycles are needed to pass the complete wavefront information, compared to 16 cycles in the non-coalesced case. Note that the largest coalescable element size is one 32-bit Data Word, DWORD (DW). VMEM instructions accessing more than 1DW/thread, such as buffer_load_dwordx2, are not coalescible.

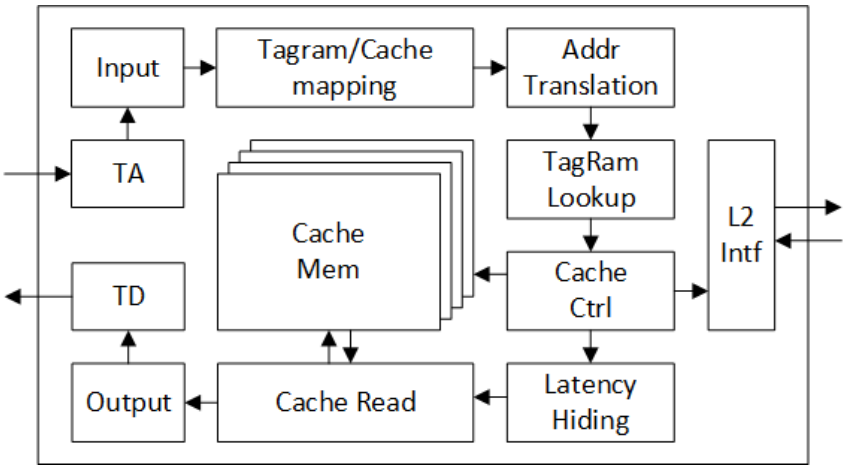


Fig. 5. MI300 GL1 Performance Model

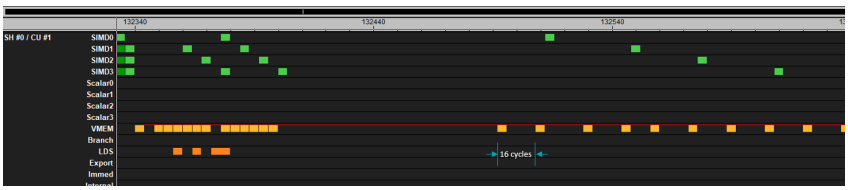


Fig. 6. TA-CU Backpressure for non-coalesced instruction burst

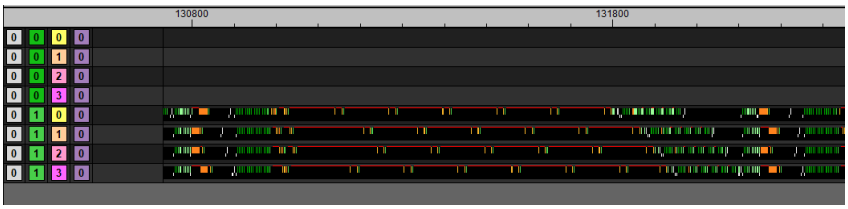


Fig. 7. Wave life with non-coalesced VMEM instructions

The address coalescing has significant impact on memory access performance. A burst of non-coalesced instructions may cause TA to be a bottleneck. The input rate of non-coalesced instructions is 4 cycles/instruction, higher than output rate of 16 cycles/instruction. A burst of 12 such instructions will overun the TA FIFO and backpressure the CU VMEM instruction issues. Figure 6 shows an example of TA-CU backpressure when a burst of non-coalescible buffer_load_dwordx2 are issued to TA. Due to the mismatched TA input rate and processing rate, the backpressure will occur quickly and the VMEM instruction issue slows down to 16 cycles/instruction, matching the TA-to-TCP transfer rate.

Figure 7 depicts the elongated wave life of a GEMM kernel resulting from the backpressure of non-coalesced VMEM instructions. The backpressure significantly reduces the GEMM processing throughput and the resulting HBM bandwidth.

Table 5. MI300 VMEM instruction bandwidth

VMEM Instruction	Coalesce	per-thread	per-instruction	per-CU Throughput
buffer_load_dword	No	4 Bytes	256 Bytes	16 bytes/cycle
buffer_load_dword	Yes	4 Bytes	256 Bytes	64 bytes/cycle
buffer_load_dwordx2	No	8 Bytes	512 Bytes	32 bytes/cycle
buffer_load_dwordx4	No	16 Bytes	1024 Bytes	64 bytes/cycle

In addition to the backpressure to CU, non-coalesced VMEM instructions will also have lower throughput, under-utilizing the available per-CU bus bandwidth. For example, the buffer_load_dwordx2 instruction accesses 8 bytes per thread, or 512 bytes/instruction. Given that it takes 16 cycles to pass the complete instruction information to TCP, the max achievable HBM bandwidth is 32 bytes/cycle for buffer_load_dwordx2 instruction. It is only 50% of the maximum per-CU bandwidth.

Table 5 shows the peak achievable HBM bandwidth when various VMEM instructions are employed. Note that, although not coalescible, the buffer_load_dwordx4 instruction can still match the per-CU bus bandwidth of 64 bytes/cycle. Besides the better bandwidth attainment, buffer_load_dwordx4, even though not coalesced, has added advantage of reduced load on tagram lookup. As a result, it is advised to employ buffer_load_dwordx4, whenever possible, to improve the attainable the per-CU and the HBM bandwidth.

3.4 TCP

TCP is constructed as physically-indexed, physically-tagged 64-way set associative cache, with cache line size of 128 bytes. A total of 4 tagrams, also called sets, are implemented, providing a total cache capacity of 32KB. Figure 5 shows the high-level TCP performance model. The main components of TCP include Input, Tagram Mapping, Virtual-to-Physical address translation, tagram lookup, cache control, latency FIFO, and GL2 interface. The following subsections highlight the main features of each component, especially those related to bandwidth attainment.

3.4.1 vL1D Cache Policy. MI300 vL1D supports two cache policies, temporal and non-temporal, to accommodate different levels of data reuse for memory read, write, and atomic operations. Non-temporal cache accesses will leave the cache lines not reusable for later operations, while temporal cache accesses would typically expect the cache lines have high locality. The cache policy hint is specified in the memory instructions Instruction Set Architecture (ISA). It determines the cache operations and cache line state maintenance, hence the resulting performance.

The cache policy affects both the cache lookup behavior and the post-lookup cache state update. Normal cache lookup allows cache hit with matching tags, enabling cache data reuse. On the other hand, forced-missed cache lookup will write back any dirty hit cache line, followed by cache line invalidation. A new cache line is then allocated for the incoming request, regardless of the cache hit/miss, essentially eliminating data reuse. The post-lookup cache state update affects how the cache line will be used. An Least Recently Used (LRU) post-lookup update will reset the cache line age to allow cache reuse. The Evict type post-lookup update will mark the cache line invalid as soon as the request for L2 is sent. This makes the cache line ready to be reclaimed as soon as the L2 response is back and the cache data is returned to CU, since the cache line is now invalid and not used by any SIMD.

Memory write operations will not reuse any available cache lines. A memory write instruction may touch several cache lines. All cache line will be lookup as Forced-Miss. The post-lookup will

Table 6. MI300 TCP Cache Policy and Behavior

Operation	Temporal	Non-temporal
Read	1. Wave/Group scope Allow cache hit LRU update 2. Device/System scope Forced-miss lookup Evict update	Forced-miss lookup Evict update
Write	Forced-miss lookup LRU update	Forced-miss lookup Evict update
Atomic	Forced-miss lookup Evict update	

update the cacheline using LRU, if the cache policy is Temporal. Non-temporal policy will evict the cache line when the L2 request is sent.

Atomic memory operations operate the same way as memory write operations with non-temporal policy. It does Forced-miss lookup and evicts the cache line when L2 request is sent.

Memory read operations are more complicated, depending on both memory accesses scopes and the cache policy. Non-temporal read performs Forced-miss cache lookup, followed by cache line evict when the line fill request is sent. The fast turn-around due to the non-temporal policy results in higher cache efficiency for workloads with no anticipated data reuse, hence higher performance.

The cache behavior for temporal memory read operations also depends on the instruction's memory access scope, specified by the ISA. An instruction's memory access scope defines 4 levels of data coherence: wave scope, group scope, device scope, and system scope, respectively. Wave scope memory accesses are coherent among all threads operating on the same CU. Group scope accesses are coherent among threads operating on the same SE. Similarly device scope access are coherent among threads operating on the same device, while system scope accesses are coherent among the same platform. The vL1D cache processes memory read with device or system scope the same as non-temporal, i.e., it does not reuse cache lines from previous instructions. It also invalidates the cache lines the corresponding L2 requests are sent. Temporal read instruction with wave or group scope accesses operates more like a typical cache. A read instruction can hit a cache line made available by previous instructions. It then updates the cache line age via LRU . It can be reclaimed when it is the oldest one and not clients are still using it. This results the cache line having extended life for potential data reuse.

Table 6 summarizes the vL1D cache policy and dynamic behavior for various scenarios.

3.4.2 Input. The TCP Input buffers memory instruction commands, addresses, and data using separate FIFOs of depths 10, 6, and 4 respectively. It back pressures TA when the its address FIFO is full, which is counted in perfmon TCP_TCP_TA_ADDR_STALL_CYCLES.

3.4.3 Tagram Mapping. The Tagram mapping block parses the incoming thread addresses from TA and maps them to one of the four cache sets. All thread addresses received in the same cycle are coalesced into unique cache lines. The resulting cache line addresses are then mapped to the cache set using hash algorithm. Set lookup serializes when two different cache lines during the same cycle are mapped to the same set. Note that thread addresses from different cycles are not coalesced.

TCP can perform up to 4 lookups per cycle, one for each set. Mapping the cache line addresses evenly to all four sets will result in higher performance. Uneven distribution of cache lines to cache set will also result in the cache sets under-utilization, or hot-spotting. In the extreme case, if all cache line lookups are directed to a single set, only 64 cache line are available to hide memory latency. Any additional cache requests will incur cache set full stall. The perfmon event TCP_ALLOC_STALL_CYCLES counts when any of the four sets are fully occupied. Any lookup stall and serialization will slow down the cache throughput.

TCP provides a cache swizzle feature to control the address to cache set mappings for Buffer instruction. The corresponding bit is set up in the Buffer Descriptor field, V#.Cache_Swizzle[2].

The set ID is determined by following algorithm.

```
setID[1:0] = func(instr_type, cache_swizzle,
                  virtual_addr[47:0], stride[13:0])
begin
    x = va » 7; // cache line aligned address
    if ( instr_type == FLAT)
        setID[1] = x[0];          setID[0] = x[1];
    else // Buffer instruction
        if (cache_swizzle ==0)
            setID[1] = x[0];      setID[0] = x[1];
        elif (stride == 256)
            setID[1] = x[1];      setID[0] = x[2] ^ x[0];
        elif (stride == 512)
            setID[1] = x[2];      setID[0] = x[3] ^ x[0];
        elif (stride == 1024)
            setID[1] = x[3];      setID[0] = x[4] ^ x[0];
        elif (stride == 2048)
            setID[1] = x[4];      setID[0] = x[5] ^ x[0];
        elif (stride == 4096)
            setID[1] = x[5];      setID[0] = x[6] ^ x[0];
        elif (stride == 8192)
            setID[1] = x[6];      setID[0] = x[7] ^ x[0];
        else // non-power-of-2 stride, or stride == 128
            setID[1] = x[0];      setID[0] = x[1];
end
```

As shown by the algorithm, for all flat instructions and buffer instruction with non-power-of-2 strides, TCP uses VA[8:7], i.e, 2 LSB bits of the cache line address, to calculate the cache set ID. Note that the setID value is bit flipped from VA[8:7]. Therefore, sequential cache line accesses map to setID 0, 2, 1, 3, and repeat the pattern.

For buffer instructions with cache swizzle enabled and the stride value is power-of-2, up to 8KB (256, 512, 1024, 2048, 4096, and 8192) , the 2 LSB bits of the stride-size aligned addresses are used to calculate the setID. For 512-byte stride, VA[10:9] are used to get the setID. The LSB of cache lined address, VA[7], is XOR'ed with VA[10:9], to further randomize the setID selection. Similar calculations apply to other stride values.

Another way to check the address to tagram mapping pattern is shown in Table 7. For the default case with no cache swizzle, VA[8:7] is mapped to setID. Sequential cache lines accesses are mapped

Table 7. MI300 TCP Tagram Mapping

VA (CL)	Default	Swizzle(256B)	Swizzle(512B)	Swizzle(1KB)	Swizzle(2KB)
0	0	0	0	0	0
1	2	1	1	1	1
2	1	2	0	0	0
3	3	3	1	1	1
4	0	1	2	0	0
5	2	0	3	1	1
6	1	3	2	0	0
7	3	2	3	1	1
8	0	0	1	2	0
9	2	1	0	3	1
10	1	2	1	2	0
11	3	3	0	3	1
12	0	1	3	2	0
13	2	0	2	3	1
14	1	3	3	2	0
15	3	2	2	3	1

to setID in cyclic sequence of 0, 2, 1, 3, and repeat. In this case, periodical addresses accesses that are 512 bytes apart will fall into the same tagram, hence hot-spotting.

Similarly for 512-byte swizzled sequential cache lines accesses, the setID exhibits the pattern of 0/2/1/3 every 4th cache line, with a period of 16 cache lines. Within the contiguous caches, the setID simply flips caused by the XOR operation with VA[7]. In this case, memory accesses with 16 cache lines apart will result in tag ram hot spotting.

In general, for swizzled cache accesses with n-KB stride, sequential memory accesses with $4n$-KB steps will result in hot spotting in the same tagram.

3.4.4 Virtual-to-Physical Address Translation. The TCP block interfaces with Unified Translation Cache - Level 1 (UTCL1) through a FIFO to provide virtual-to-physical address translation. Two probing interfaces are implemented to allow a maximum of 2 translation lookups per cycle. During each cycle, up to 4 translation requests could be requested from the 4 tagrams. When a single tagram issues a translation request, it will always goes to probe 0. When two tagrams have translation requests, they will be directed to probe 0 and probe 1, respectively. When 3 or 4 tagrams have translation requests, they will be serialized on each probing interface to UTCL1. This serialization will slow down the address translation throughput. The occurance can be measured using perfmon event TCP_UTCL1_SERIALIZATION_STALL.

On MI300, the two UTCL1 interfaces work in lock-step fashion. TCP can send requests to UTCL1 only if both interfaces are ready to accept transactions. TCP will be stalled by UTCL1 due to FIFO full, or either of the interface is not ready. This stall event will eventually backpressure TCP, then TA and CU.

3.4.5 Tagram Lookup. All four sets operate in parallel, providing a maximum cache lookup rate of 4 lookups per cycle. All lookup misses are transferred to the cache controller block for request merging and eventually sent to the L2 interface for cache line fill from the next level cache. The cache controller can also receive up to 4 miss requests per cycle from the tag ram block. The request

rate from vL1D to the L2 cache is limited to 1 request per cycle. The cache controller block interfaces with a latency hiding First In First Out (FIFO) buffer (LFIFO) to hide the round-trip latency from the L2 cache. Enough outstanding L2 requests can be buffered in the FIFO to avoid vL1D being stalled. The cache controller also ensures that the LFIFO has available buffer before sending requests to the L2 interface. The cache controller is also responsible for writing data from the incoming write requests into the cache RAM.

Once the return cache lines from L2 is written to the cache RAM, the corresponding pending requests in the Latency hiding FIFO (LFIFO) is popped out to the cache read block. The cache read block resolves cache RAM bank conflicts and issues read requests. For global memory reads, the cache RAM read responses are passed to TD for format change and transfer to shader side VGPRs accordingly. For global memory writes, the cache read block transfers the read data to the L2 interface, instead.

3.4.6 Cache Controller. TCP controller is responsible for accumulating LFIFO entries and pushing them into LFIFO. For write and atomic operations, it performs write combining into cache lines and sends the write requests to TCP memory blocks. All tag ram lookup misses are forwarded to RFIFO in the memif block.

The TCP controller is stalled when one of the following conditions is asserted:

(1) LFIFO is full
(2) RFIFO is full
(3) TCP memory block write is stalled due to read data return from TCR
(4) TCP memory block write is stalled due to empty write data FIFO on the input side

The controller stall will further back pressure the tag RAM block.

3.4.7 Latency FIFO. TCP is equipped with a 704-deep LFIFO to hide memory access latency. Both TCP lookup hit and lookup miss requests are buffered in the LFIFO, until the return data is ready. For memory write operations, each entry stores 13-16 cache lines to implement instruction level write combinations. An LFIFO entry is consumed when 13 cache lines have already been accumulated. This is to avoid entry overflow, since we have 4 tag lookups per cycle and waiting for another cycle may result in 17 cache lines in total. An LFIFO entry can accommodate a maximum of 16 cache lines for write operation.

The LFIFO entry accumulates differently for memory read operations. The LFIFO occupancy is closely related to TA buffer coalescing and memory access granularity. Each entry accumulates up to 16 DW worth of data. Note that TA does not coalesce DWORDx2 and DWORDx4 read. It processes 4 threads per cycle and send them to TCP. All DWord worth data request in a cycle will be store in LFIFO. Each memory read instruction consumes 16 LFIFO entries. For DWordx4 read, each cycle requests 16 DWord data, hence each LFIFO entry represents 16 "lanes." Similarly, each LFIFO entry buffers 8 "lanes" for DWordx2, and 4 "lanes" for Dwordx1, respectively. Table 8 lists the LFIFO sizing and the maximum number of cache lines it can accommodate for various memory access types. A perfmon event of perf_tcp_lfifo_stall_cycles will increment when the LFIFO is full.

3.4.8 L2 Interface. TCP interfaces with L2 using a 64-deep Request FIFO (RFIFO). It can buffer up to 4 TCP miss requests per cycles while the request rate to L2 is 1 request per cycle. The L2 read and write request sizes are 128-byte and 64-byte, respectively. The backpressure from L2 to TCP can be measured using TCP_TCR_TCP_STALL_CYCLES, which is a good indicator of TCP saturating the L2 cache. A high L2-TCP backpressure rate could result in lower per-CU bandwidth.

Table 8. MI300 LFIFO sizing

Transaction	LFIFO Entry Size	Entries/Instr	Cache lines accommodated
DWordx1 Write	2 CL	1	1,408
DWordx2 Write	4 CL	1	2,816
DWordx4 Write	8 CL	1	5,632
DWordx1 Read	4 DW	16	88
DWordx2 Read	8 DW	16	176
DWordx4 Read	16 DW	16	352

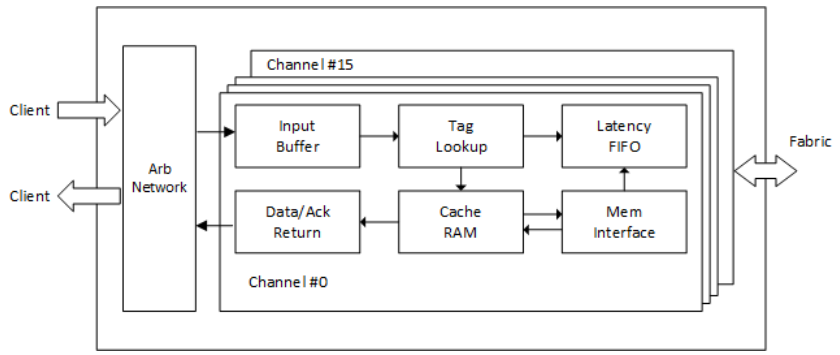


Fig. 8. MI300 GL2 Performance Model

3.5 Vector-L2 Cache

The L2 cache is the central hub on XCD servicing memory access requests from all clients, including CU, instruction cache, Scalar Level-1 Data Cache (sL1D), UTCL2, etc. It is built as a 16-channel writeback and write-allocate cache. Each channel operates independently as a 128-set 16-way set associative cache with cache line size of 128 bytes. The aggregated L2 cache capacity is 4MB per XCD. The major components of the L2 cache include the frontend arbitration network, input buffer, tag lookup and cache memory, LFIFO, SOC fabric interface, and the read data and write ACK return interfaces, as shown in Figure 8.

3.5.1 L2 Cache Policy. The L2 cache implements three cache policies: LRU, Streaming, and Bypass, respectively. The LRU policy operates as typical cache does. It allows cache hit and the cache line's age is reset when accessed. The Streaming policy has the identical behavior as LRU, except that the cache line age is not updated. This leaves the cache line aged out quickly, making it available for reclaim, as soon as it is not in use. The fast cache line turn-around due to the Streaming policy improves performance for workloads with little data reuse, since there is more chance for an incoming request to acquire a cache line. The Bypass policy enforce no cache reuse. A hit cache line will be written back, if dirty, followed by invalidation. The cache line is then fetched from external memory and temporarily stored in the cache RAM. However, it will be invalidated as soon as the data is returned to clients.

The cache policy is determined by the access scope, the temporal attribute, and the compute partition modes. Both access scope and the temporal attribute are derived from the memory access instruction. The compute partition mode is from the system configuration.

Table 9. MI300 L2 Cache Policy and Behavior

Operation	Temporal	Non-temporal
Read Write	1. Wave/Group scope: LRU 2. Device scope CPX: LRU SPX/DPX: Bypass 3. System scope: Bypass	1. Wave/Group scope: Streaming 2. Device scope CPX: Streaming SPX/DPX: Bypass 3. System scope: Bypass
Atomic	Bypass	

MI300 supports three compute partition modes, Single Partition Accelerator (SPX), Dual Partition Accelerator (DPX), and Core Partition Accelerator (CPX), respectively, to group XCDs into logical GPU devices. The SPX mode aggregates all XCDs into one device sharing a single coherence domain. On the other hand, the DPX mode partitions the XCDs into two logical device, while CPX results in a configuration with one XCD per device, each having its own coherence domain.

For temporal memory read/write, the L2 cache operates using the normal LRU policy for Wave scope and Group scope instructions, as well as Device scope instructions on a CPX node. Otherwise, it operates as Bypass model. Similarly, for non-temporal memory read/write, the L2 cache operates as Streaming for Wave/Group scope, and Device scope on a CPX node. Bypass policy is resorted to for System scope operations and Device scope operations on SPX/DPX nodes.

Table 9 lists the detailed L2 policy and behavior for all memory accesses under with temporal and non-temporal hints.

3.5.2 Arbitration Network. The frontend arbitration network interconnects all L2 clients and the L2 channels via separate read and write buses with widths of 128 bytes and 64 bytes, respectively. Each client interfaces with one of the 16 pairs of buses, both can be active in the same cycle. A client's memory access request is routed via mapping logic to the destination channel. The arbitration enforces proper burst control and fairness among all clients. The aggregated interface bandwidths are 2048 bytes/cycle (read) and 1024 bytes/cycle (write). Each L2 channel, however, services 1 request per cycle from any client.

The interconnect performs arbitration for read, write requests and data, and read/write return data or acknowledgment separately. The arbitration efficiency directly affects the achievable L2 access performance. Table 10 lists the per-XCD L2 cache bandwidths for all MI300 SOCs, normalized to bytes/cycle.

3.5.3 Input Buffer. The input buffer accepts memory access requests destined to the target channel and prepare for cache lookup. For write and atomic accesses, the request and data are transferred asynchronously. The input buffer initiates the request to the source client for data. The winning clients from the requesting channel can then send the 64-byte data. The arrival data are also buffered in an FIFO for late processing. At any cycle, a client can win data write to a single L2 channel. Collectively, the L2 cache can process up to 16 read or write requests simultaneously.

3.5.4 LFIFO. The GL2 cache has a shared LFIFO pool of 512 deep to hide memory latency. In addition, each GL2 client has a 7-deep FIFO to allow parallel accesses.

3.5.5 Fabric Interface. The fabric interface is the primary demarcation between GFXIP and SoC. Each GL2 channel independently interfaces with SoC via a 64-byte bus, shared by both read and write. The aggregated GL2-SoC bandwidth is thus 1024 bytes/cycle. Each interface has separate buffers for read and write requests. The SoC will back pressure GL2 when the corresponding buffer

Table 10. MI300 L2 Bandwidth

Metric	Unit	MI300A	MI300X	MI308	MI350
#CUs/XCD		38	38	20	32
Read Bus	Bytes	128	128	128	128
Write Bus	Bytes	64	64	64	64
Read Arb Efficiency		90%	90%	90%	90%
Write Arb Efficiency		58%	58%	58%	75%
L2 Read BW (peak)	Bytes/cycle	2048	2048	2048	2048
L2 Read BW (eff)	Bytes/cycle	1843	1843	1843	1843
L2 Read BW/CU	Bytes/cycle	48.5	48.5	92.2	57.6
L2 Write BW (peak)	Bytes/cycle	1024	1024	1024	1024
L2 Write BW (eff)	Bytes/cycle	594	594	594	768
L2 Write BW/CU	Bytes/cycle	15.6	15.6	29.7	24

is depleted, slowing down the GL2 operation. A workload can be categorized as memory bandwidth bound when significant SoC side stall is encountered.

4 MI300 BANDWIDTH ATTAINMENT ANALYSIS

4.1 Perfmons and Performance Metrics

The MI300 performance analysis heavily relies on the ROCProfiler tool[4] and the hardware performance events/metrics[3]. The comprehensive profiling allows thorough understanding of the workload characteristics and the possible bottlenecks. Each silicon supports hundreds perfmon events to provide visibility into each IP block's dynamic behavior. At first glance, the large number of provided events and metric could be daunting to new users and a brief tutorial is needed. The available events are grouped into several categories, including

(1) Clocking
(2) Workload characterization
(3) IP architectural performance
(4) IP micro architecture performance

The clocking related perfmon events count the elapsed cycles at IP and SoC level. Both free running cycles and busy cycles are supported. The free running cycles measure the duration when an IP is enabled, while the busy cycles measure the duration when an IP is busy processing at least one request. The clocking events provide good "wall clock" to derive various performance metrics. They also indicate the utilization of an IP or the GPU as a whole, i.e., the percentage time the IP is busy processing requests. For example, GRBM_GUI_COUNT is a free running clock when the GPU is enabled, while GRBM_GUI_ACTIVE counts the number of cycle the GPU is occupied with at leave on wave.

The events for workload characterization provide (mostly) deterministic measurement of the workload execution. This set of events include instruction mix counting, requests counting at each IP interface. The instruction mix measurement reveals the workload characteristics, whether dominantly compute or memory accesses, without checking the disassembly. The request counting, e.g., number of 32-byte/64-byte/128-byte requests, enable clear view of traffic flow between IP blocks. They can be also used to derive the bandwidth on the IP interfaces.

The IP architecture performance related events and metrics allow evaluation of the IP performance and possible bottlenecks. The cache hit/miss rates at each memory heirarchy and the stall events

between IPs fit that category. The inter-IP stall rates could quickly help narrow down the bottleneck identification. For example, the lack of any back pressure from L2 to vL1D could eliminate GL2 as a limiting factor, and one shall focus on vL1D and upper stream blocks.

The IP micro architecture related events may seem a little obscure to new analyst. Solid understanding the GPU performance model is required to properly understand the significance of these events. However, the careful analysis of the micro-arch related perfmon events and metrics could lead to quick root cause identification of the workload under test. These events sometimes serve as "secret sauce" for performance analysis. Some examples include TCP_TAGRAM[0..3]_REQ and TCP_ALLOC_STALL_CYCLES. They could help diagnosis TCP capacity stall and tagram hot-spotting related issues.

For reference, Table 12 and 13 list the key performance events and metrics used in this document.

4.2 Performance Analysis

A workload running on MI300 could be limited by multiple factors, namely, compute bound, memory bandwidth bound, memory latency bound, and kernel dispatch bound. A compute bound kernel exhibits itself as saturating the compute pipelines, e.g., VALU. A memory bandwidth bound kernel saturates the realistically achievable HBM bandwidth. On the other hand a memory latency bound kernel will cause the wave fronts to be parked up, unable to fill the deep memory pipe. Finally, a dispatch bound kernel will not be able to dispatch kernels as fast as possible due to lack of various compute resources. At any stage of optimization, one limiting factor will dominate while others still play a role in affecting the target performance. The software optimization could transform the workload into one dominated by different limit factors. This section intends to sketch some actionable recipes to help diagnosis the issue and speed up the optimization.

For ease of disposition, Figure 9 provides a diagnosis tree to sketch the main steps involved in MI300 performance analysis, especially for bandwidth limited workloads. It also aims to provide some actionable performance analysis methodology that a user could systematically follow. However, it is not expected that this tree is the sole process to follow to identify bottlenecks for all workloads. Fundamentally, performance analysis is more an art, rather than a mechanical dictionary lookup based problem solving.

4.2.1 Roofline Characterization. All performance analysis starts with the initial assessment of workload under testing, including achieved FLoating-point Operations Per Second (FLOPS) and memory bandwidth. These metrics can sometimes be acquired without resorting to ROCProf. This is mostly the case for GEMM micro benchmarks, the number of floating point and integer operations and the amount of memory accesses are know a prior. This document assumes the approach of profiling based measurement.

The achieved memory bandwidth can be measured on the GL2-SoC interface, by counting transactions of different sizes during the life of the kernel execution. MI300 memory read requests can be of 32-byte, 64-byte, and 128-byte, while the memory write request sizes are of 32-byte and 64-byte, respectively. Metric TCC_EA_RDREQ_BW and TCC_EA_WRREQ_BW (Table 13) calculate the read and write traffic. The total amount of traffic is simply the sum of the two. One can derive the aggregated bandwidth in GB/s or Bytes/cycle by normalizing it with the corresponding wall clocks. Also note that the measured bandwidth represents total traffic flowing across the demarcation interface between GFXIP and SoC. It consists of HBM traffic, system memory traffic, and XGMI traffic. The metrics calculate the HBM bandwidth when it is indeed the main traffic type. MI350 also enables additional perfmon events to measure bandwidth to/from individual target.

MI300 provides perfmon events to measure GL2-SoC read/write latency (Metric:TCC_EA_RD_LATENCY, TCC_EA_WR_LATENCY).

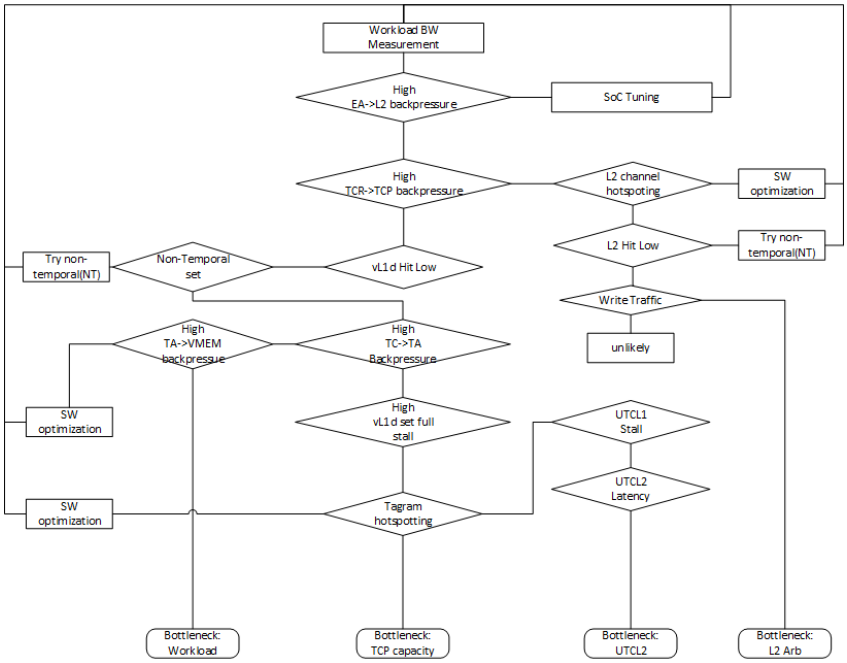


Fig. 9. MI300 Bandwidth Diagnosis Chart

4.2.2 GL2 Performance Analysis. The GL2 perfmon events allow characterizing incoming traffic on the GL1-GL2 interface, the GL2 cache dynamic behavior, and GL2-SoC interactions. It also allow measuring the traffic unbalancing among the 16 GL2 channel.

On the GL1-GL2 interface, one can measure the traffic breakdown based on traffic direction(Read, Write, Atomic), Cache policy type (LRU/Non-temporal), or traffic MType(UC/NC/CC/RW). The traffic breakdown provides a clear characterization of the workload and points to potential optimizations. For streaming type applications that little locality is expected, setting the cache policy to non-temporal could improve the performance due to the improved cacheline reuse. A key GL2 metric, TCP_TCR_TCP_STALL_RATE, measures the percentage of time that GL2 stalls GL1 requests due to lack of buffer resources. A high GL2-GL1 stall rate, e.g., over 10%, could clearly indicate that GL2 is a potential bottleneck. For example, the GL2 arbitration network efficiency is 58% on MI300 and 75% on MI350. The flooded write traffic from GL1 could result in the high GL2-GL1 stall, since the arbitration network cannot keep up with the incoming requests.

All GL2 requests are routed from clients, including GL1, instruction cache and scalar data cache, to one of the 16 GL2 channel. Uneven traffic distribution could incur GL2 channel hot spotting, hence the reduced performance. To identify the potential channel unbalancing, one can resort to per-channel perfmon events and metrics to analyze the per-channel requests, enabled by the per-channel hardware perfmon events. As a baseline, the aggregated traffic measurement provides the initial workload assessment.

The GL2 cache dynamic behavior includes cache hit/miss and cache utilization, measurable via TCC_HIT_RATE and TCC_UTIL, respectively. These metrics characterize workload's memory access locality and the percentage of time the GL2 cache is busy working on cache requests. A high

GL2 hit rate could significantly improve the workload performance due to the reduced memeory access latency. Note that streaming type workload has inherent low cache hit rate.

The GL2 cache lookup can be stalled by downsteam back pressure on the GL2-SoC interface, or micro architecture resource limits. TCC_TAG_STALL_RATE is such a key metric providing the lumpsum of all internal stalls. Table 11 details the contributors to TCC_TAG_STALL. A high TCC_TAG_STALL could indicate that GL2 is back pressured by SoC and causes the slowdown of cache lookups.

Table 11. TCC Tag stall (perf_tag_stall) breakdown

Stall	**Reason**	Note
Tag Request Stall	EA Read FIFO full	Read FIFO: 2-deep (non-bypass: 14-deep)
	EA Writeback FIFO full	Writeback FIFO: 2-deep
	TCC cache set full	All 16 ways in the set are in use
	TCC LFIFO FIFO full	FIFO to LFIFO: 2-deep
	TCC uncached write/atomic FIFO full	uncached write/atomic FIFO: 2-deep
	EA rdreq HBM credit depletion	credits:64
	EA rdreq GMI credit depletion	credits:64
	EA rdreq IO credit depletion	credits:72
	RW probed_in_use stall	
	TCC uncached write/atomic ack entry stall	
	TCC cache bypass stall	The selected way is in use or way pending writes, or the way is hit but dirty
	TCC probe FIFO full	Probe FIFO: 512-deep

On the GL2-SoC interface, one can measure traffic breakdown, memory access bandwidth, and memory access latency. The high memory bandwidth, e.g., over 80%, in conjunction with TCC_TAG_STALL_RATE, could indicate a workload being memory bandwidth bound.

4.2.3 GL1 Performance Analysis. GL1 affects workload performance in many important ways. Similar to GL2, one can analyze the GL1 performance on SQ-GL1 interface, GL1 cache dynamics, and GL1-GL2 interface. Each GL1 also interfaces with a dedicated UTCL1 to enable logical-to-physical address translation.

SQ-GL1 Interface. The key metrics on the SQ-GL1 interfaces include traffic type breakdown (flat vs. buffer), traffic direction breakdown (read, write, atomic) and GL1-SQ back pressures on the command, address, and data buses. A high back pressure rate, e.g., over 10%, could indicate GL1 and/or GL2 are potential bottlenecks. The traffic breakdown over either type (flat/buffer) or direction (read/write/atomic) also provides assessment of the workload implementation and the dominant memory accesses.

GL1 Cache Dynamics. As discussed in Section 3.4, there are four major components within GL1: TA, TD, TCP, and UTCL1. The important performance metrics include address coalescing, virtual-to-physical address translation, cache lookup and latency hiding. The close analysis could reveal if GL1 is a bottleneck limiting the workload performance, or it is further limited by GL2.

Metric TA_BUF_ADDR_COALESCE_RATE measures the TA's capability in coalescing Buffer or Flat addresses. The optimal rate of 100% indicates that all 64 threads' address can be transmitted to TCP in 4 cycles. The worst case rate is 25%, requiring 16 cycles to transfer addresses for all threads. Combined with static code analysis, one may identify the inefficiency in software implementation.

For example, buffer_load_dwordx2 shall be replaced with buffer_load_dwordx4, if all possible, to boost the memory access efficiency.

On the TA-TCP interface, the key metric TCP_TA_ADDR_STALL_RATE allows identifying whether TCP is a bottleneck. It is a metric aggregating all impacts from TCP, including address translation, cache lookup, and GL2-GL1 stall. Any stalls in TCP will eventually show up in this metric. A high stall rate could potentially point to TCP's limited capability in latency hiding.

TCP's cache lookup performance can be examined from several aspects: per-CU bandwidth, cache hit rate, cache access policy, tagram access balancing, tagram capacity stalls, and UTCL1 address translation.

The per-CU bandwidth, measurable from TCP_TCC_BW, normalized by the number of active CUs, provides a good indicator of the GPU performance scalability. A good bandwidth limited GEMM kernel could reach over 50%, or 32 bytes/cycle.

The cache hit rate and cache access policy reveal the workload characteristics. High cache hit rate will greatly improve the workload performance due to the significantly reduce memory access latency. The cache access policy is a hint from memory access instructions, indicating the temporal attribute. For non-temporal workload which has little memory access locality, setting the cache policy to NT would also improve the system performance, benefiting from better cache line reuse. Note that the TCP cache hit rate is correlated with TA's address coalescing and cache line length. All thread addresses transferred in the same cycle are consolidated into unique cache line addresses to carry out the cache lookup. For sequential memory accesses using buffer_load_dwordx4, 16 cycles are needed to transfer all addresses from TA, each cycle representing a 64-byte request. Since TCP cache line is 128-byte wide, each cycle accesses half cache line, i.e., cycle 0 accesses cache line0 byte 63-0, cycle 1 access cache line 0 byte 127-64, etc. This will result in 50% cache hit rate.

TCP has a relatively small cache of 256KB split into 4 tagrams, achieving a max of 4 lookups per cycle. Spreading cache requests evenly into all 4 tagrams will improve the system performance and increase TCP's latency hiding capability. When a tagram is unable to completely hide latency, i.e., all cache lines in a tagrams are used up, TCP capacity stall could incur. This can be caused by either high burst rate from SQ, or long experienced GL2 latency. The TCP capacity stall is measurable via TCP_SET_FULL_STALL_RATE. The consequence is that it back pressures UTCL1, since it cannot retrieve the translated physical address in time. This, in return, back pressures TA which is reflected in TCP_TA_ADDR_STALL_RATE. Once TA is backup, it will further back pressures SQ, observable from software as SQ_VMEM_CMD_STALL_RATE. The end effect is that software experiences long VMEM latency, long wave life, and the ultimate long kernel execution time, hence the lower performance.

The unbalanced tagram accesses increases the chance of TCP capacity stall. Although it is not directly measurable, the tagram hotspotting can be inferred from TCP misses from these tagrams, measurable as TCP_TAGRAM[0..3]_REQ. Assuming the four tagrams have similar cache hit rate, the unbalanced TCP miss traffic can point to tagram hotspotting, when ratio between the hottest and coldest tagram accesses is more than five times.

The UTCL1 address translation impact on the workload performance is also reflected via TCP_TA_ADDR_STALL_RATE. Similar as TCP, UTCL1 also employs cache lookup. However, A simple measurement of UTCL1 translation hit rate alone may not reveal all performance issues, since most workloads' UTCL1 hit rate is close to 97%. Both UTCL1 hit/miss latency and request rate play important role in affecting the performance. A high incoming burst request rate could result in UTCL1 stall due to reaching the max inflight count, which is observable from TCP_UTCL1_STALL_INFLIGHT_MAX. On other hand, any UTCL1 non-duplicate miss, which is measurable via TCP_UTCL1_STALL_LFIFO_NOT_RES, will trigger UTCL2 lookup, hence slow down the UTCL1 lookup. In the best case of UTCL2 hit, the round-trip latency from UTCL2 is

around 82 cycles. When UTCL2 misses, a high page walk penalty of several hundred cycles will incur, causing even more UTCL1 request backup. The high TCP_UTCL1_STALL_LFIFO_NOT_RES could potentially point to UTCL1 as a bottleneck. Software optimization could help alleviate the issue. For example, for a workload accessing the same amount of memory, employing dwordx4 instruction could reduce the number of UTCL1 lookups, compared to dwordx2 instruction.

GL1-GL2 Interface. The GL1-GL2 interface is a demarcation between GL1 and GL2 cache in diagnosis the workload bottleneck. The measured interface bandwidth indicates CU's performance scalability, while the interface stall rate could help isolate the bottleneck. The lack of GL2-GL1 stall could clearly point to GL1 as the primary suspect for additional optimization.

4.2.4 Compute Performance Analysis. This document primarily focuses on MI300 attainable memory bandwidth and analysis. The detailed evaluation of the compute unit deserves a separate document. The coverage in this section provide just enough details understanding a workload's characteristics related to compute bound and memory bandwidth bound.

The workload behavior can be gauged from its wave life distribution, wave instruction issue, and its interactions with LDS and VMEM.

A wave's life is divided into three phases: instruction dependency resolution, instruction issue waiting, and instruction issue/execution. The dependency resolution waiting consists of cycles where the wave spends for various conditions to be met, before scheduling the next instruction. For example, a wave may wait for barrier or waitcnt to be zero. This can be measured by Once the dependency is resolved, the wave will compete with other waves in the SIMD for the target execution pipelines, e.g., VALU. The winner wave's instruction will then be issued and executed. MI300 provides perfmon events and metrics to measure the average wave life, as well as cycles spent in each phases:

(1) SQ_WAVE_LIFE
(2) SQ_WAVE_WAIT_DEPENDENCY
(3) SQ_WAIT_WAIT_ISSUE
(4) SQ_WAVE_ISSUE_EXEC

Additional perfmon events are available to measure the cycles spent waiting for individual pipelines, including

(1) SQ_WAIT_INST_VALU
(2) SQ_WAIT_INST_LDS
(3) SQ_WAIT_INST_VMEM
(4) SQ_WAIT_INST_SCA

A typical memory latency bound kernel exhibits as spending high percentage of time waiting for dependency resolution. When an execution pipeline(VALU/LDS/VMEM/SCA) is saturated, a wave will spend more time waiting for instruction issues, i.e., SQ_WAIT_WAIT_ISSUE. The instruction issue wait for each pipeline will further indicate if the workload is dominantly limited by VALU, LDS, VMEM, or SCA. For example, signifcant waiting time for VALU issue could hint that the workload is compute bound. Similar profiling can indicate if a a workload is limited by LDS, VMEM, or Scalar pipelines. For a VMEM bound workload, the measurement on the SQ-TA interface could determine the level of back pressure from the cache subsystem.

4.2.5 Wave Dispatch Performance Analysis. The SPI wave dispatch efficiency directly impacts the compute unit's wave concurrency, memory latency hiding, and even VALU utilization. For workloads whose performance benefit from high wave occupancy, e.g., BabelStream, the SPI analysis could reveal software optimization opportunities.

	A	B
1	Dispatch_ID	20
2	GPU_ID	2
3	Queue_ID	1
4	PID	432
5	TID	432
6	Grid_Size	131,072
7	Workgroup_Size	256
8	LDS_Per_Workgroup	27,648
9	Scratch_Per_Workitem	0
10	Arch_VGPR	112
11	Accum_VGPR	128
12	SGPR	64

Fig. 10. Wavefront resource usage

SPI wave dispatch is resource constrained, a wave is not ready for dispatch until the resource requirements for all waves in the same workgroup are satisfied. For each managed resource listed in Table 4, MI300 provides a corresponding perfmon to the resource depletion occurrence. For example, SPI_RA_VGPR_SIMD_FULL_CSN accumulates every cycles the number of SIMDs whose VGPRs are fully allocated and cannot accommodate a new wave. Similarly, SPI_RA_LDS_CU_FULL_CSN accumulates every cycles the number of CUs which do not have LDS for new workgroup. The comparison of absolute counter values between different depletion occurrences may not make sense, give that they have different units. However, the dominantly high counting of one category, e.g., LDS depletion, points to the limiting resource affecting the wave dispatch.

The per-workgroup/wave resource usage is captured by ROCProf. A sample profiling output is shown in Figure 10. Note that on M300, the VGPR usage is sum of archVGPR and accumVGPR. In this example, a total 240 VGPRs are used. As such, with a max of 512 VGPRs, each SIMD can accommodate 2 waves to satisfy the VGPR constraint. Similarly, the workgroup's LDS consumption is 27KB, resulting in a max of 2 workgroups/CU. Given that the workgroup size is 4 waves/group, the max wave occupancy is 2 waves/SIMD and 8 waves/CU to meet all constraints.

5 SUMMARY

This document provides the high level view of GFXIP performance models for MI300 family SOCs, including MI300A, MI300X, MI308, and MI350X. It aims to illustrate both theoretical and attainable memory bandwidths at each memory hierarchy. The detailed software visible performance models equips engineers with better understanding of the underlying limiting factors for the workload under optimization. The actionable performance analysis methodology, along with the customer visible perfmon events and metrics, are also shared to help systematic workload characterization and optimization on AMD GPUs.

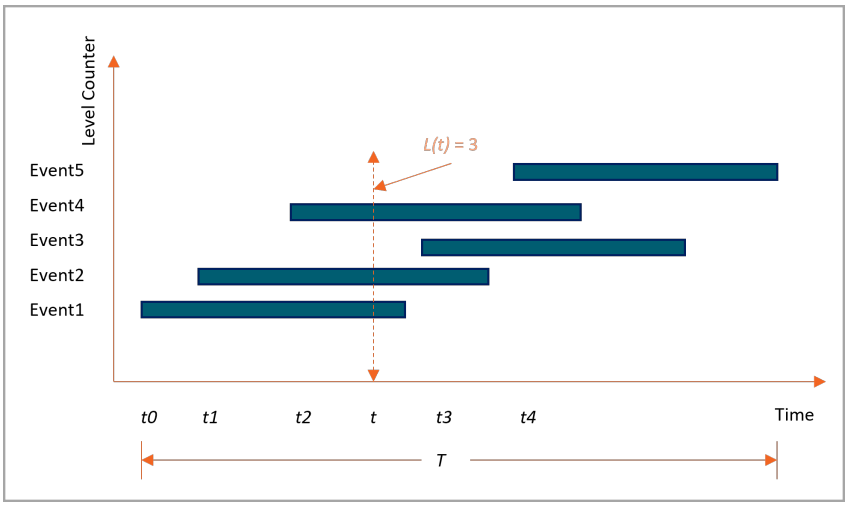


Fig. 11. Level Counter measurement

REFERENCES

[1] AMD. 2023. *AMD CNDA3™ Architecture - The All-New AMD GPU Architecture for the Modern Era of HPC and AI.* https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf

[2] AMD. 2023. *AMD Instinct MI300 Instruction Set Architecture Reference Guide.* https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/amd-instinct-mi300-cdna3-instruction-set-architecture.pdf

[3] AMD. 2024. *MI300 and MI200 series performance counters and metrics.* https://rocm.docs.amd.com/en/latest/conceptual/gpu-arch/mi300-mi200-performance-counters.html

[4] AMD. 2024. *ROCProfiler Documentation.* https://rocm.docs.amd.com/projects/rocprofiler/en/latest/

[5] Jack Dongarra and AI Geist. 2022. Report on the Oak Ridge National Laboratory's Frontier System. ICL-UT-22-05, Knoxville, TN.

[6] HPCWire.com. 2022. *AMD's MI300 APUs to Power Exascale El Capitan Supercomputer.* https://www.hpcwire.com/2022/06/21/amds-mi300-apus-to-power-exascale-el-capitan-supercomputer/

[7] Xiaomin Lu and Noah Wolfe. 2023. *Hierachical Roofline Performance Analysis on AMD Instinct MI200 GPU.* https://sc23.conference-program.com/presentation/?id=tut116&sess=sess224

[8] Alan Smith and Norman James. 2022. AMD Instinct™ MI200 Series Accelerator and Node Architectures. In *2022 IEEE Hot Chips 34 Symposium (HCS).* 1–23. https://doi.org/10.1109/HCS55958.2022.9895477

[9] TOP500.org. 2023. *TOP500, The list, June 2023.* https://www.top500.org/lists/top500/2023/06/

A PERFORMANCE EVENTS

Many perfmon events are available on MI300 to diagnosis the HBM bandwidth attainment and to identify the related limiting factor. Table 12 lists the commonly used ones.

B PERFORMANCE METRICS

2.5pt

C LEVEL COUNTERS

C.1 Design

GPU performance analysis frequently requires checking latency and occupancy related metrics. Some examples are average wavefront occupancy and memory access latency. A generic mechanism, level counting, is designed on MI GPUs to allow measurement of these derived metrics.

Table 12. MI300 Perfmon Events for HBM Bandwidth Attainment diagnosis

0	IP Block	Perfmon	Unit	Definition
1	GRBM	GRBM_GUI_ACTIVE	Cycles	# of cycles GPU subsystem is busy
2	SQ	SQ_VMEM_TA_ADDR_FIFO_FULL	Cycles	# of cycles TA address FIFO is full
3	SQ	SQ_VMEM_TA_CMD_FIFO_FULL	Cycles	# of cycles TA command FIFO is full
4	SQ	SQ_VMEM_TA_DATA_FIFO_FULL	Cycles	# of cycles TA data FIFO is full
5	SQ	SQ_INSTS_VMEM	#	# of VMEM instructions issued
6	SQ	SQ_INSTS_VMEM_RD	#	# of VMEM Read instructions issued
7	SQ	SQ_INSTS_VMEM_WR	#	# of VMEM Write instructions issued
8	SQ	SQ_WAIT_INST_VMEM	quad-cycle	# of quad-cycles waiting for VMEM instruction issue
9	SQ	SQ_WAVE_CYCLES	quad-cycle	# of quad-cycles spent by waves in the CUs
10	SQ	SQ_WAVES	#	# of dispatched wavefronts
11	SQ	SQ_WAIT_ANY	quad-cycle	Number of quad-cycles spent waiting for anything
12	SQ	SQ_WAIT_INST_ANY	quad-cycle	Number of quad-cycles spent waiting for any instruction issue
13	SQ	SQ_WAIT_INST_LDS	quad-cycle	Number of quad-cycles spent waiting for LDS issue
14	SQ	SQ_WAIT_INST_VALU	quad-cycle	Number of quad-cycles spent waiting for VALU issue
15	SQ	SQ_WAIT_INST_VMEM	quad-cycle	Number of quad-cycles spent waiting for VMEM issue
16	SQ	SQ_WAIT_INST_SCA	quad-cycle	Number of quad-cycles spent waiting for SALU/SMEM issue
17	SQ	SQ_ACTIVE_INST_ANY	quad-cycle	Number of quad-cycles each wave is working on an instruction.
18	SQ	SQ_ACCUM_PREV_HIRES	#	Level counter accumulator
19	SQ	SQ_LEVEL_WAVES	#	Level counter for # of active waves
20	SQ	SQ_INST_LEVEL_VMEM	#	Level counter for # of inflight VMEM instructions
21	TA	TA_ADDR_STALLED_BY_TC_CYCLES	Cycles	# of cycles TA addr path is stalled by TC
22	TA	TA_DATA_STALLED_BY_TC_CYCLES	Cycles	# of cycles TA data path is stalled by TC
23	TCP	TCP_TCP_TA_ADDR_STALL_CYCLES	Cycles	# of cycles TCP stalls TA addr interface
24	TCP	TCP_TCP_TA_DATA_STALL_CYCLES	Cycles	# of cycles TCP stalls TA data interface
25	TCP	TCP_RFIFO_STALL_CYCLES	Cycles	# of cycles memorey request FIFO accesses stalled
26	TCP	TCP_ALLOC_STALL_CYCLES	Cycles	# of cycles at least one tagram is full
27	TCP	TCP_TCR_TCP_STALL_CYCLES	Cycles	# of cycles TCP is stalled by TCR
28	TCP	TCP_PENDING_STALL_CYCLES	Cycles	# of cycles TCP stall due to data pending from L2
29	TCP	TCP_TOTAL_ACCESSES	#	# of unique thread addresses passed from TA to TCP
30	TCP	TCP_TOTAL_CACHE_ACCESSES	#	# of TCP cache lookups
31	TCP	TCP_TAGRAM0_REQ	#	# of TCP lookup misses from Tagram0
32	TCP	TCP_TAGRAM1_REQ	#	# of TCP lookup misses from Tagram1
33	TCP	TCP_TAGRAM2_REQ	#	# of TCP lookup misses from Tagram2
34	TCP	TCP_TAGRAM3_REQ	#	# of TCP lookup misses from Tagram3
35	TCP	TCP_UTCL1_REQUEST	#	# of UTCL1 address translation requests
36	TCP	TCP_UTCL1_TRANSLATION_HIT	#	# of UTCL1 translation hits
37	TCP	TCP_UTCL1_TRANSLATION_MISS	#	# of UTCL1 translation misses
38	TCP	TCP_UTCL1_STALL_INFLIGHT_FULL	Cycles	# of cycles UTCL1 stalls due to max inflight count
39	TCP	TCP_UTCL1_STALL_LFIFO_NOT_RES	Cycles	# of cycles UTCL1 stalls due to requested page not present
40	TCP	TCP_UTCL1_LFIFO_FULL	Cycles	# of cycles UTCL1 latency FIFO full
41	TCP	TCP_TCC_READ_REQ	#	# of L2 Read requests
42	TCP	TCP_TCC_WRITE_REQ	#	# of L2 Write requests
43	TCP	TCP_TCC_ATOMIC_REQ	#	# of L2 Atomic requests
44	TCP	TCP_TCC_READ_REQ	#	# of L2 Read requests
45	TCC	TCC_REQ	#	# of L2 requests
46	TCC	TCC_READ	#	# of L2 Read requests
47	TCC	TCC_WRITE	#	# of L2 Write requests
48	TCC	TCC_ATOMIC	#	# of L2 Atomic requests
49	TCC	TCC_STREAMING_REQ	#	# of L2 streaming requests
50	TCC	TCC_HIT	#	# of L2 lookup hits
51	TCC	TCC_MISS	#	# of L2 lookup misses
52	TCC	TCC_TAG_STALL	Cycles	# cycles L2 request pipeline stalls
53	TCC	TCC_EA0_RDREQ	#	# SOC Read requests, including 32Byte, 64Byte, and 128Byte
54	TCC	TCC_EA0_RDREQ_32B	#	# 32-byte SOC Read requests
55	TCC	TCC_BUBBLE	#	# 128-byte SOC Read requests
56	TCC	TCC_EA0_WRREQ	#	# SOC Write requests, including 32Byte and 64Byte
57	TCC	TCC_EA0_WRREQ_64B	#	# 64-byte SOC Write requests
58	TCC	TCC_EA0_RDREQ_LEVEL	#	Accumulated # of inflight EA Read requests
59	TCC	TCC_EA0_WRREQ_LEVEL	#	Accumulated # of inflight EA Write requests
60	SPI	SPI_RA_TMP_STALL_CSN	Cycles	# of cycles temp space does not fit
61	SPI	SPI_RA_WAVE_SIMD_FULL_CSN	SIMD-cycles	Sum of SIMDs with wave slot fully occupied
62	SPI	SPI_RA_VGPR_SIMD_FULL_CSN	SIMD-cycles	Sum of SIMD with VGPR fully occupied
63	SPI	SPI_RA_SGPR_SIMD_FULL_CSN	SIMD-cycles	Sum of SIMD with SGPR fully occupied
64	SPI	SPI_RA_LDS_CU_FULL_CSN	CUs	Sum of CUs with LDS fully allocated
65	SPI	SPI_RA_BAR_CU_FULL_CSN	CUs	Sum of CUs with all barriers allocated
66	SPI	SPI_RA_BULKY_CU_FULL_CSN	CUs	Sum of CU with BULKY used

Table 13. MI300 Perfmon Metrics

0	IP Block	Metric	Unit	Definition
1	GRBM	GPU_WALL_CLK	Cycles	GRBM_GUI_COUNT
2	GRBM	GPU_BUSY	Cycles	GRBM_GUI_ACTIVE
3	SQ	SQ_WAVE_LIFE	Cycles	4 * SQ_WAVE_CYCLES
4	SQ	SQ_VMEM_CMD_STALL_RATE		SQ_VMEM_TA_CMD_FIFO_FULL / SQ_BUSY
5	SQ	SQ_VMEM_ADDR_STALL_RATE		SQ_VMEM_TA_ADDR_FIFO_FULL / SQ_BUSY
6	SQ	SQ_VMEM_DATA_STALL_RATE		SQ_VMEM_TA_DATA_FIFO_FULL / SQ_BUSY
7	SQ	SQ_VMEM_WAIT_RATE		SQ_WAIT_INST_VMEM / SQ_WAVE_CYCLES
8	SQ	SQ_VMEM_LATENCY	Cycles	SQ_INST_LEVEL_VMEM / SQ_INSTS_VMEM
9	TA	TA_BUF_ADDR_COALESCE_RATE		4 * TA_BUFFER_WAVEFRONTS / TA_BUFFER_TOTAL_CYCLES
10	TCP	TCP_BUSY	Cycles	TCP_GATE_EN2
11	TCP	TCP_MISS	#	TCP_TAGRAM0_REQ + TCP_TAGRAM1_REQ + TCP_TAGRAM2_REQ + TCP_TAGRAM3_REQ
12	TCP	TCP_HIT	#	TCP_TOTAL_CACHE_ACCESSES - TCP_MISS
13	TCP	TCP_REQ_FORCE_MISS_EVICT_RATE	#	(TCP_TOTAL_MISS_EVICT_READ + TCP_TOTAL_MISS_EVICT_WRITE) / TCP_TOTAL_ACCESSES
14	TCP	TCP_HIT_RATE		TCP_HIT / TCP_TOTAL_CACHE_ACCESCES
15	TCP	TCP_TCC_RD_BW	Bytes	128 * TCP_TCC_READ_REQ + 64 * TCP_TCC_ATOMIC_WITH_RET_REQ
16	TCP	TCP_TCC_WR_BW	Bytes	64 * TCP_TCC_WRITE_REQ + 64 * TCP_TCC_ATOMIC_WITHOUT_RET_REQ
17	TCP	TCP_TCC_BW	Bytes	TCP_TCC_RD_BW + TCP_TCC_WR_BW
18	TCP	TCP_TCC_REQ_RATE		TCP_MISS / TCP_BUSY
19	TCP	TCP_SET_FULL_STALL_RATE		TCP_ALLOC_STALL_CYCLE / TCP_BUSY
20	TCP	TCP_TCR_TCP_STALL_RATE		TCP_TCR_TCP_STALL_CYCLES / TCP_BUSY
21	TCP	TCP_TA_ADDR_STALL_RATE		TCP_TCP_TA_ADDR_STALL_CYCLES / TCP_BUSY
22	TCP	TCP_TA_DATA_STALL_RATE		TCP_TCP_TA_DATA_STALL_CYCLES / TCP_BUSY
23	TCP	TCP_LFIFO_STALL_RATE		TCP_LFIFO_STALL_CYCLES / TCP_BUSY
24	TCP	TCP_RFIFO_STALL_RATE		TCP_RFIFO_STALL_CYCLES / TCP_BUSY
25	TCP	TCP_TCP_TD_STALL_RATE		TCP_TD_TCP_STALL_CYCLES / TCP_BUSY
26	UTCL1	UTC1_MISS_RATE		TCP_UTCL1_TRANSLATION_MISS / TCP_UTCL1_REQUEST
27	UTCL1	UTCL1_LATENCY	Cycles	TCP_CLIENT_UTCL1_INFLIGHT / TCP_UTCl_REQUEST
28	UTCL1	UTCL1_UTCL2_LATENCY	Cycles	TCP_UTCL1_UTCL2_INFLIGHT / TCP_UTCl_TRANSLATION_MISS
29	UTCL1	UTCL1_INFLIGHT_STALL_RATE		TCP_UTCL1_STALL_INFLIGHT_MAX / TCP_BUSY
30	UTCL1	UTCL1_LFIFO_FULL_STALL_RATE		TCP_UTCL1_LFIFO_FULL / TCP_BUSY
31	UTCL1	UTCL1_THRASH_STALL_RATE		TCP_UTCL1_THRASHING_STALL / TCP_BUSY
32	UTCL1	UTCL1_LFIFO_NOT_RES_STALL_RATE		TCP_UTCL1_STALL_LFIFO_NOT_RES / TCP_BUSY
33	TCC	TCC_STREAM_REQ	#	TCC_STREAMING_REQ
34	TCC	TCC_LRU_REQ	#	TCC_REQ - TCC_STREAMING_REQ[5]
35	TCC	TCC_EA_RDREQ_64B	#	TCC_EA0_RDREQ - TCC_EA0_RDREQ_32B - TCC_BUBBLE
36	TCC	TCC_EA_RDREQ_BW	Bytes	32*TCC_EA0_RDREQ_32B + 64*TCC_EA_RDREQ_64B + 128*TCC_BUBBLE
37	TCC	TCC_EA_WRREQ_BW	Bytes	32*(TCC_EA0_WRREQ - TCC_EA0_WRREQ_64B) + 64*TCC_EA0_WRREQ_64B
38	TCC	TCC_TAG_STALL_RATE		TCC_TAG_STALL / TCC_BUSY
39	TCC	TCC_HIT_RATE		TCC_HIT / (TCC_HIT + TCC_MISS)
40	TCC	TCC_EA_RD_LATENCY	Cycles	TCC_EA0_RDREQ_LEVEL / (TCC_EA0_RDREQ + TCC_BUBBLE)
41	TCC	TCC_EA_WR_LATENCY	Cycles	TCC_EA0_WRREQ_LEVEL / TCC_EA0_WRREQ
42	TCC	TCC_IB_STALL_RATE		TCC_IB_STALL / TCC_BUSY

As shown in Figure 11, multiple occurrences of some perfmon event can happen at the same time, e.g., multiple outstanding LDS instructions are issued. Two interesting derived metrics about the perfmon event are:

- Average duration of an perfmon event (Latency).
- Average outstanding occurrences (Occupancy).

For wavefront execution on CUs, the corresponding metrics are:

- Average wavefront life.
- Average number of active wavefronts on the Shader Engine.

Level counters are designed for these purposes. A level counter measures the value of the event at a sampling instant. For example, SQ_LEVEL_WAVES measures the number of inflight wavefronts at sampling instant *t*. In Figure 11, there are 3 active events at time *t*, hence the corresponding level counter value is 3.

A separate hardware accumulator[6], SQ_ACCUM_PREV_HIRES, is always paired with the level counter.

[6]There are actually two accumulators: SQ_ACCUM_PREV_HIRES and SQ_ACCUM_PREV. One accumulates the level counter every cycles, while the other accumulates every fourth cycle. This document uses high resolution one by default.

Table 14. Level Counters

ID	Metric	Counter	Level Count	Accumulator
0	instrFetchLatency	sq_ifetch	sq_ifetch_level	sq_accum_hires
1	vmemLatency	sq_insts_vmem	sq_insts_level_vmem	sq_accum_hires
2	smemLatency	sq_insts_smem_norm	sq_insts_level_smem	sq_accum_hires
3	ldsLatency	sq_insts_lds	sq_insts_level_lds	sq_accum_hires
4	waveLife	sq_waves	sq_level_waves	sq_accum_hires

Assume the level counter value at time t is $L(t)$, the final value of the accumulator is simply the occurrence-time integration of the profiled hardware event:

$$I(T) = \int_0^T L(t)\,dt,$$

Where T is the profiling duration.

When either the profiling duration T, or the total occurrences N are known, which are usually available from separate performance counters, then

- Average wavefront life = $L(T)/N$
- Average wavefront occupancy = $L(T)/T$

The same mechanism is employed to acquire other performance metrics, e.g., memory latency.

C.2 Usage

On AMD Instinct™ GPUs, the level counter must be supplied in pair with the corresponding accumulator counter during the profiling to get the proper measurement. Either accumulator counter, SQ_ACCUM_PREV or SQ_ACCUM_PREV_HIRES, can be used. For example, in order to measure SQ's instruction fetch latency, the following line shall be specified in the configuration file:

```
pmc: SQ_IFETCH                SQ_IFETCH_LEVEL                SQ_ACCUM_HIRES
```

The average instruction fetch latency, in Cycles, can then be calculated as

$$\text{instrFetchLatency} = \frac{\text{SQ_ACCUM_HIRES}}{\text{SQ_IFETCH}}$$

Table 14 lists all the level counters and the corresponding accumulators used to profile the Compute Unit related performance metrics.

D MULTI-INSTANCE COUNTERS

A general note on multi-instance aggregated counter using the underlying hardware counters is as follows.

The vector L1 cache subsystem and L2 cache consist of multiple instances and/or multiple channels. Each Compute Unit has a dedicated vector L1 cache subsystem, including Texture Addressing Unit, Texture Data Unit and Vector L1D cache. Each vector L1D cache related performance counter aggregates values across all eight shader engines with the same CU ID, creating a total of 16 instances. Meanwhile, the L2 cache consists of 32 physical channels, each collecting performance values independently, hence 32 performance monitor instances.

The aggregation of a performance counter over all instances is generally used to analyze the system performance. The derived metrics of general form <base_hardware_counter>_sum are thus used to denote those aggregations. For example, the derived counter of tcc_hit_sum is the L2 cache hit count aggregated over all L2 cache channels. All aggregated counters are available in the XML configuration file.

This document freely references these aggregated counters without individual clarification.